

# IoT-Enabled Retail Analytics on Azure Databricks

## Resources Creation

### creation of storage account:

The screenshot displays the Azure portal interface for a storage account named 'sagarcatalog'. The left sidebar shows the 'Storage accounts' list with 'sagarcatalog' selected. The main content area shows the 'Overview' tab for the storage account. Key details include:

- Resource group:** LabsKraft2027
- Location:** westus2
- Subscription:** LabsKraft
- Subscription ID:** d008c7cc-9795-4292-bf79-74ae52cda63d
- Disk state:** Available
- Created:** 8/25/2025, 9:00:00 AM
- Performance:** Standard
- Replication:** Locally-redundant storage (LRS)
- Account kind:** StorageV2 (general purpose v2)
- Provisioning state:** Succeeded

The 'Properties' tab is active, showing 'Data Lake Storage' settings: Hierarchical namespace (Enabled), Default access tier (Hot), and Blob anonymous access (Disabled). The 'Security' section shows 'Require secure transfer for REST API operations' and 'Storage account key access' both enabled.

### creation of access connector:

The screenshot displays the Azure portal interface for an access connector named 'sagarconnector'. The left sidebar shows the 'Access Connect...' list with 'sagarconnector' selected. The main content area shows the 'Overview' tab for the access connector. Key details include:

- Resource group:** LabsKraft2027
- Location:** West US 2
- Subscription:** LabsKraft
- Subscription ID:** d008c7cc-9795-4292-bf79-74ae52cda63d
- State:** Succeeded
- Resource ID:** /subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/...

The 'Overview' tab is active, showing 'Access control (IAM)' and 'Tags' sections. The 'Settings' and 'Automation' sections are also visible.

giving access connector permission for storage:

Microsoft Azure

Search resources, services, and docs (G+/I)

Copilot

fredence14@michaelklo...  
DEFAULT DIRECTORY (MICHAELK...

Home > Storage accounts > sagarstorage0 | Access Control (IAM) >

Add role assignment ...

Role

Members

Conditions

Review + assign

Showing a filtered list of roles because your permissions include a condition. [Learn more](#)  
[View my access](#)

Selected role

Storage Blob Data Contributor

Assign access to

User, group, or service principal

Managed identity

Members

+ Select members

| Name                | Object ID | Type |
|---------------------|-----------|------|
| No members selected |           |      |

Description

Optional

Review + assign

Previous

Next

Select managed identities

Some results might be hidden due to your ABAC condition.

Subscription \*

LabsKraft

Managed identity

Access Connector for Azure Databricks (1)

Search by name

sagarcconnector

/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/LabsKra...

Selected members:

No members selected. Search for and add one or more members you want to assign to the role for this resource.

[Learn more about RBAC](#)

Select

Close

Feedback

creation of uc:

Microsoft Azure

databricks

Search data, notebooks, recents, and more...

CTRL + P

tred\_d4

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

AI/ML

Playground

Experiments

Features

Catalog

Serverless Starter Warehouse Serverless S

Type to search...

My organization

tred\_d4

system

sagar\_cat\_project1

default

dit\_output

information\_schema

sagar\_catalog

sagar\_catalog2

sagar\_catalog3

pratcatalog

pratcatalog1

sahil\_catalog

sahil\_catalog\_new

sanyam\_catalog

sanyam\_new\_catalog

Delta Shares Received

samples

Legacy

hive\_metastore

Catalog Explorer

sagar\_cat\_project1

Share

Create schema

Overview

Details

Permissions

Workspaces

About this catalog

Metastore Id

4f3b2018-fdc7-47a3-965c-db7d2dc65f0b

Created At

Aug 25, 2025, 09:13 AM

Created By

tredence14@michaelkloudkraft@hotmail.onmicrosoft.com

Updated At

Aug 25, 2025, 09:13 AM

Updated By

tredence14@michaelkloudkraft@hotmail.onmicrosoft.com

Storage Root

abfss://raw0@sagarcatalog.dfs.core.windows.net/

Storage Location

abfss://raw0@sagarcatalog.dfs.core.windows.net/\_unitystorage/catalogs/00d8640c-c495-4e5e-b9c6-447107a58745

Advanced

Predictive Optimization

ENABLED (Inherited)

## creation of namespace and eventhub:

[Home](#) > [Event Hubs](#) >

 **Create Namespace** ...

Event Hubs

**Project Details**

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription \*

LabsKraft

Resource group \*

LabsKraft2027

[Create new](#)

**Instance Details**

Enter required settings for this namespace, including a price tier and configuring the number of units (capacity).

Namespace name \*

sagarspace ✓  
.servicebus.windows.net

Region \*

West US 2

 The region selected supports Availability zones. Your namespace will have Availability Zones enabled. [Learn more.](#)

Pricing tier \*

Basic

[Browse the available plans and their features](#)

Throughput Units \*

1

[Review + create](#)

[< Previous](#)

[Next: Advanced >](#)

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

tredence14@michaelklo...  
DEFAULT DIRECTORY (MICHAELKLO...)

Home > Event Hubs >

Create Namespace

Event Hubs

Validating...

Basics Advanced Networking Tags Review + create

Event Hubs Namespace  
by Microsoft

Basics

|                                      |               |
|--------------------------------------|---------------|
| Namespace name                       | sagarhub      |
| Subscription                         | LabsKraft     |
| Resource group                       | LabsKraft2027 |
| Region                               | West US 2     |
| Pricing tier                         | Basic         |
| Throughput Units                     | 1             |
| Availability Zones (Zone Redundancy) | Enabled       |

Networking

|                     |               |
|---------------------|---------------|
| Connectivity method | Public access |
|---------------------|---------------|

Security

|                     |     |
|---------------------|-----|
| Minimum TLS version | 1.2 |
|---------------------|-----|

Create < Previous Next >

24°C  
Mostly cloudy

Search

ENG  
IN

10:48  
25-08-2025

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

tredence14@michaelklo...  
DEFAULT DIRECTORY (MICHAELKLO...)

Home > Event Hubs > sagarspace >

Create Event Hub

Event Hubs

Validation succeeded.

Basics Capture Review + create

Event Hubs Instance  
by Microsoft

Basics

|                 |          |
|-----------------|----------|
| Name            | sagarhub |
| Partition count | 1        |

Retention

|                      |        |
|----------------------|--------|
| Cleanup policy       | Delete |
| Retention time (hrs) | 6      |

Capture

|                |               |
|----------------|---------------|
| Capture Status | Not Supported |
|----------------|---------------|

Create < Previous Next >

# creating synapse workspace:

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

DEFACTO GROUP  
DEFAULT DIRECTORY

Home > Azure Synapse Analytics >

Create Synapse workspace ...

Validation succeeded

information with the provider(s) of the offering(s) for support, billing and other transactional activities. Microsoft does not provide rights for third-party offerings. For additional details see [Azure Marketplace Terms](#).

Basics

Subscription

Resource group

Region

Workspace name

Data Lake Storage Gen2 account

Data Lake Storage Gen2 file system

Managed resource group

Role assignments

LabsKraft

LabsKraft2027

West US 2

(new) sagars

https://sagarstorage0.dfs.core.windows.net

raw

None

The Storage Blob Data Contributor role will be assigned on the specified Data Lake Storage Gen2 account to both the workspace managed identity and the current user.

Security

Authentication method

SQL Server admin login

SQL Password

Double encryption

Use both local and Microsoft Entra ID authentication

sqladminuser

\*\*\*\*\*

No

Create

< Previous

Next >

Download a template for automation

# setting up synapse ->

linking my adls for storing old data and these can be pulled into azure dbx

Microsoft Azure

Synapse Analytics

sagars

DEFACTO GROUP  
DEFAULT DIRECTORY

Synapse live

Validate all

Publish all

Data

Workspace

Linked

Filter resources by name

Azure Data Lake Storage Gen2 2

sagars (Primary - sagarstorage0)

raw (Primary)

(Attached Containers)

raw

Other users in your workspace may have access to modify this item. Do not use this item unless you trust all users who may have access to the workspace.

New SQL script

New notebook

New data flow

New integration dataset

Upload

Download

New folder

More

raw

Name

Last Modified

Content Type

Size

retail\_iot\_data.csv

8/25/2025, 10:26:07 AM

130.8 KB

## creation of new dedicated pool:

**Microsoft Azure** | Synapse Analytics | sagars

Synapse live | Validate all | Publish all

SQL pools

The serverless SQL pool, Built-in, is immediately available.

+ New Refresh

Filter by name

Showing 1-1 of 1 items (1 Serverless, 0 Dedicated)

Name

Built-in

### New dedicated SQL pool

Validation succeeded.

Basics \* Additional settings \* Tags **Review + create**

#### Product details

Azure Synapse Analytics dedicated SQL pool by Microsoft

Terms of use | Privacy policy

Est. cost per hour: 99.83 INR

[View pricing details](#)

#### Terms

By clicking "Create", I (a) agree to the legal terms and privacy statement(s) associated with the Marketplace offering(s) listed above; (b) authorize Microsoft to bill my current payment method for the fees associated with the offering(s), with the same billing frequency as my Azure subscription; and (c) agree that Microsoft may share my contact, usage and transactional information with the provider(s) of the offering(s) for support, billing and other transactional activities. Microsoft does not provide rights for third-party offerings. For additional details see [Azure Marketplace Terms](#)

#### Data source

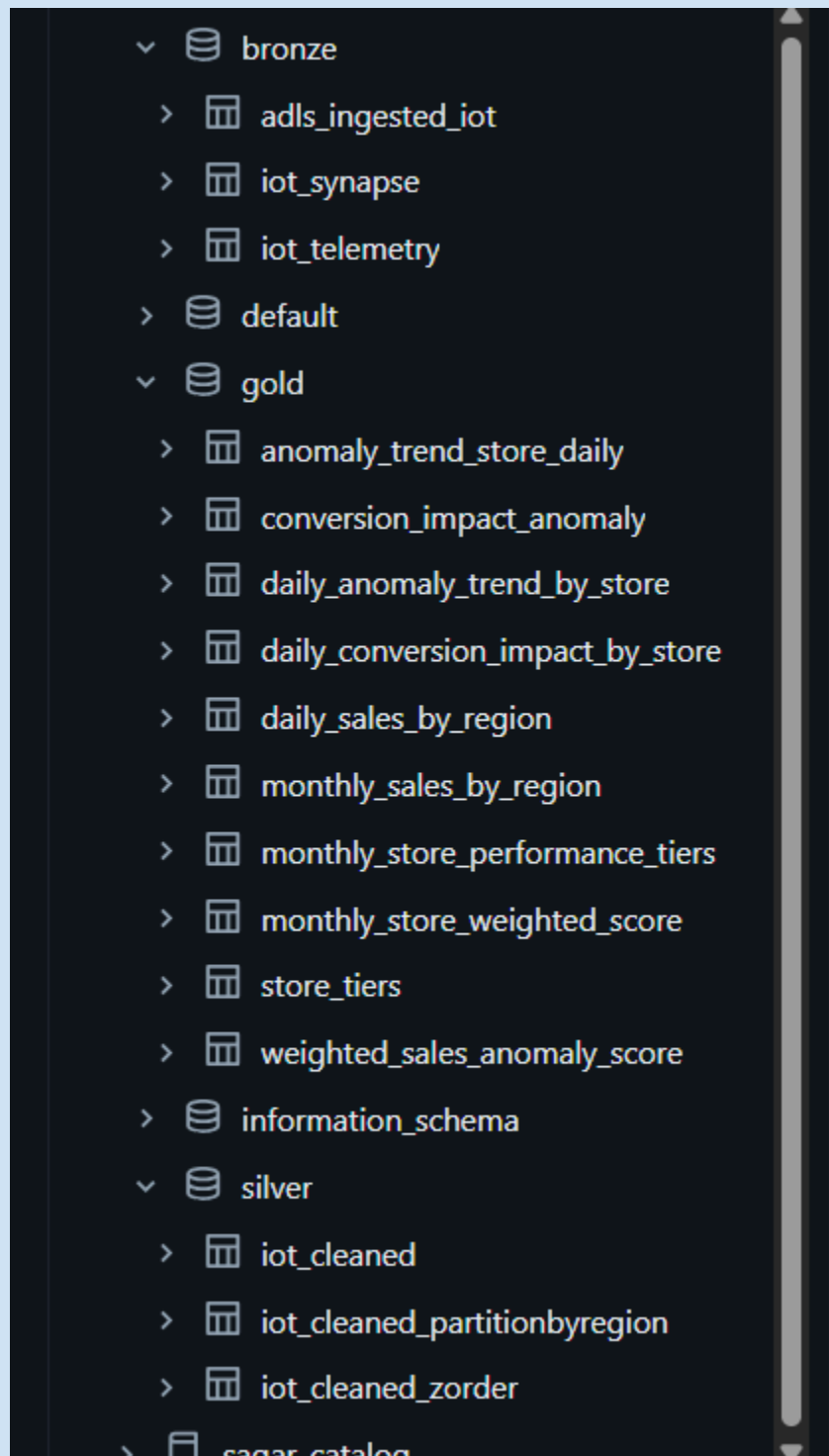
|                         |        |
|-------------------------|--------|
| Dedicated SQL pool name | db1    |
| Performance level       | DW100c |

#### Additional settings

|                   |                              |
|-------------------|------------------------------|
| Use existing data | Blank                        |
| Collation         | SQL_Latin1_General_CP1_CI_AS |

**Create** < Previous Download template for automation Cancel

Quick Summary->



## **Ingestion mode:**

### **Bronze Layer Ingestion – Summary**

#### **1. ADLS Gen2:**

- CSV read from ADLS (`retail_iot_data.csv`)
- Stored as Delta table `adls_ingested_iot` in `bronze` schema

#### **2. Event Hub:**

- File uploaded to VM → sent to Event Hub
- Spark structured streaming reads, parses JSON
- Stored as `iot_telemetry` table in bronze

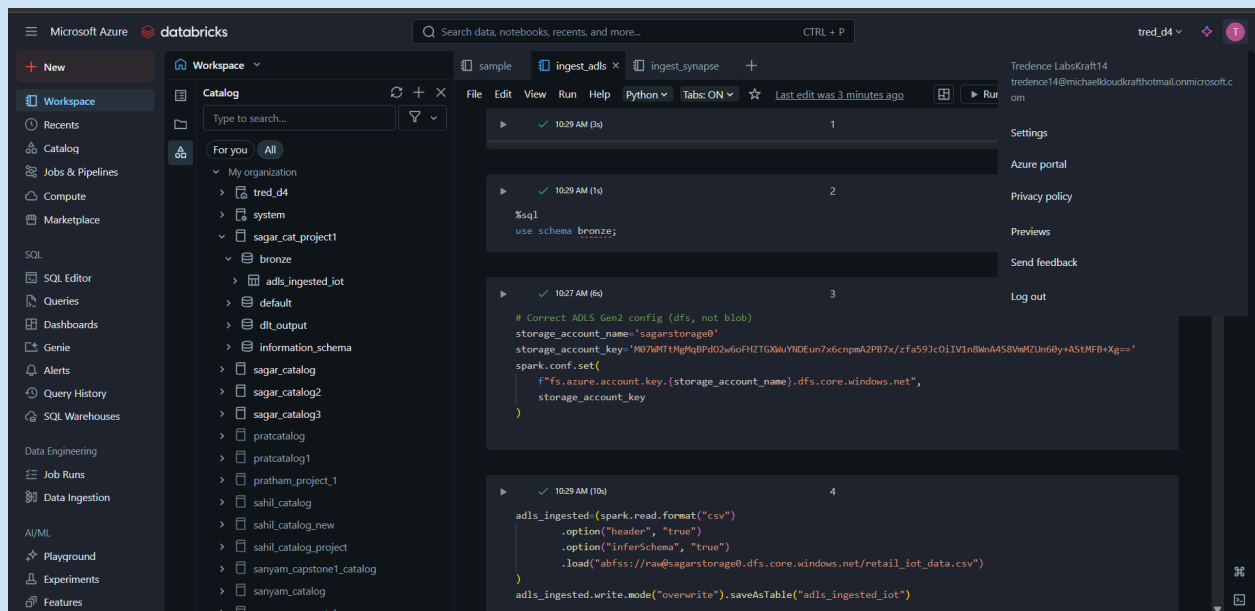
#### **3. Azure Synapse:**

- Synapse table read via JDBC connector
- Loaded and stored as Delta table `iot_syn` in bronze

#### **i) ADLS**

adls connection and storing file in `sagar_cat_project1.bronze` schema





# MAGIC %sql

# MAGIC use catalog sagar\_cat\_project1;

# MAGIC create schema if not exists bronze;

# MAGIC %sql

# MAGIC use schema bronze;

# Correct ADLS Gen2 config (dfs, not blob)

storage\_account\_name="

storage\_account\_key="

spark.conf.set(

f'fs.azure.account.key.{storage\_account\_name}.dfs.core.windows.net",

storage\_account\_key

)

spark.sql("DROP TABLE IF EXISTS adls\_ingested\_iot")

adls\_ingested = (

spark.read.format("csv")

.option("header", "true")

.load("abfss://raw@sagarstorage0.dfs.core.windows.net/retail\_iot\_data.csv")

)

adls\_ingested.write.mode("overwrite").saveAsTable("adls\_ingested\_iot")

display(adls\_ingested)

adls\_ingested\_table->

```
.option("inferSchema", "true")\
.load("abfss://raw@sagar-storage0.dfs.core.windows.net/retail_iot_data.csv")\
)\
adls_ingested.write.mode("overwrite").saveAsTable("adls_ingested_iot")\
display(adls_ingested)
```

(7) Spark Jobs

adls\_ingested: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 12 more fields]

|    | OrderID                              | CustomerID | Region | StoreID   | Product    | Quantity | UnitPrice | SalesAmount | OrderTime                     | DeviceID |
|----|--------------------------------------|------------|--------|-----------|------------|----------|-----------|-------------|-------------------------------|----------|
| 1  | db07ec0d-e867-408a-846e-c2d330484d4b | CUST-2824  | East   | Store-009 | Mobile     | 2        | 322.1     | 644.2       | 2023-01-05T23:24:55.000+00:00 | Temp_Sc  |
| 2  | 93d89258-6ef2-4bb8-9a71-cb590cb558b5 | CUST-7912  | East   | Store-001 | Laptop     | 2        | 503.69    | 1007.38     | 2023-01-30T05:24:09.000+00:00 | RFID_Sc  |
| 3  | f97636e9-3dab-48ad-aad4-0eca034d3111 | CUST-9928  | South  | Store-008 | Tablet     | 5        | 592.47    | 2962.35     | 2023-01-01T07:34:17.000+00:00 | RFID_Sc  |
| 4  | f20eae76-3e09-4580-8429-af1d6b76209a | CUST-3547  | West   | Store-011 | Laptop     | 1        | 790.86    | 790.86      | 2023-01-18T10:14:28.000+00:00 | POS_Ter  |
| 5  | 5dbcc9df-0275-4427-a4b6-089b43465200 | CUST-8527  | East   | Store-013 | Laptop     | 11       | 621.7     | 6838.7      | 2023-01-18T13:20:07.000+00:00 | Temp_Sc  |
| 6  | c6bd3963-d239-42b7-b414-4e39289bc2da | CUST-5741  | East   | Store-008 | Printer    | 1        | 791.25    | 791.25      | 2023-01-23T00:15:40.000+00:00 | Temp_Sc  |
| 7  | 02fadf02-d15e-4527-94b1-9cb915a71209 | CUST-6820  | West   | Store-009 | Smartwatch | 1        | 1237.81   | 1237.81     | 2023-01-09T07:23:27.000+00:00 | Temp_Sc  |
| 8  | 90e7e190-9d09-47a3-a399-98562cf389dc | CUST-7216  | North  | Store-018 | Mobile     | 3        | 1693.56   | 5080.68     | 2023-01-03T17:10:20.000+00:00 | RFID_Sc  |
| 9  | 50f6c840-47fc-4117-a776-78701a911f99 | CUST-7572  | North  | Store-003 | Mobile     | 5        | 1758.92   | 8794.6      | 2023-01-16T06:37:34.000+00:00 | RFID_Sc  |
| 10 | 7784cd70-6019-4be0-bce4-51626ae52101 | CUST-8517  | West   | Store-009 | Mobile     | 2        | 1502.73   | 3005.46     | 2023-01-27T03:57:14.000+00:00 | POS_Ter  |
| 11 | 6d65d67c-bc5e-43bf-9522-50e645fa58f8 | CUST-7543  | North  | Store-008 | Mobile     | 5        | 1012.36   | 5061.8      | 2023-01-03T06:53:40.000+00:00 | RFID_Sc  |
| 12 | d6c5b3f1-076e-4c24-a836-318d6588eedd | CUST-7916  | East   | Store-013 | Tablet     | 5        | 1992.44   | 9962.2      | 2023-01-26T16:27:46.000+00:00 | POS_Ter  |
| 13 | 1817588f-6017-413a-9e20-c876857ee103 | CUST-1188  | East   | Store-018 | Printer    | 3        | 1548.77   | 4646.31     | 2023-01-17T12:19:59.000+00:00 | RFID_Sc  |
| 14 | 54a4533d-e78f-4d3d-9426-8eb38292bbd3 | CUST-1053  | North  | Store-017 | Printer    | 2        | 1039.98   | 2079.96     | 2023-01-06T03:58:39.000+00:00 | Temp_Sc  |

ii)Event Hub:

we will connect to vm and then send data from local machine to eventhub which at the end will be recieved by DBX

creation of vm (namespace and eventhub already created) ->

Microsoft Azure

Home > Compute infrastructure | Virtual machines >

Create a virtual machine

Validation passed

Preferred e-mail address:

Preferred phone number:

You have set SSH port(s) open to the internet. This is only recommended for testing. If you want to change this setting, go back to Basics tab.

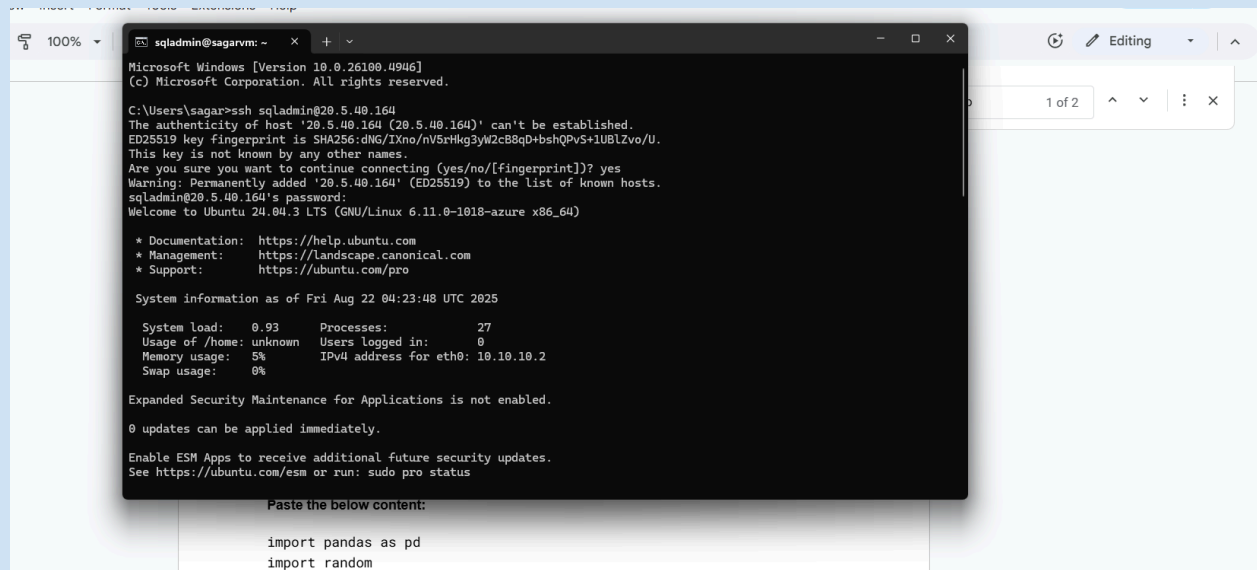
Basics

|                      |                                 |
|----------------------|---------------------------------|
| Subscription         | LabsKraft                       |
| Resource group       | LabsKraft2027                   |
| Virtual machine name | sagarvm                         |
| Region               | Australia East                  |
| Availability options | Availability zone               |
| Zone options         | Self-selected zone              |
| Availability zone    | 1                               |
| Security type        | Trusted launch virtual machines |
| Enable secure boot   | Yes                             |
| Enable vTPM          | Yes                             |
| Integrity monitoring | No                              |

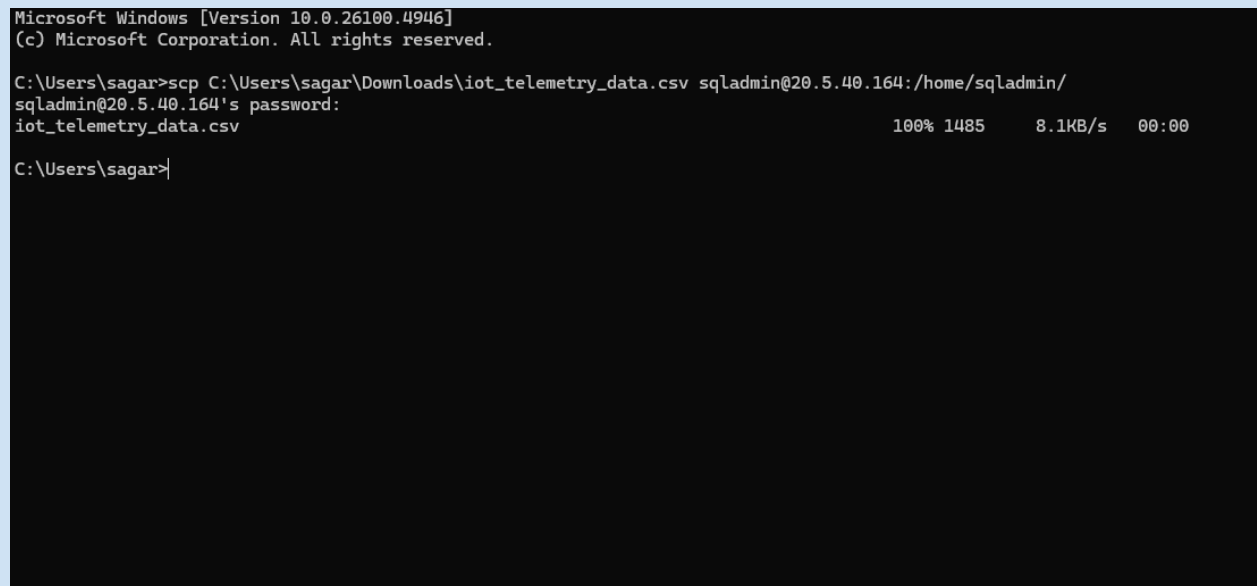
< Previous Next > Create

Download a template for automation Give feedback

setting up vm session on local machine->



uploading my file to vm which is considered for ingestion



nano [send.py](#)

```
GNU nano 7.2 send.py
ls /home/sqladmin/ from azure.eventhub import EventHubProducerClient, EventData
import pandas as pd
import time
import json

# Load the CSV
df = pd.read_csv('hot_telemetry_data.csv')

# Event Hub connection
connection_str = "Endpoint=sb://sagarspace.servicebus.windows.net/;SharedAccessKeyName=RootManageSharedAccessKey;SharedAccessKey=2v87F8+tBbuM02M5qjzsPlUAVWbYgfeZ+AEhBrynDE="
eventhub_name = "sagarhub"

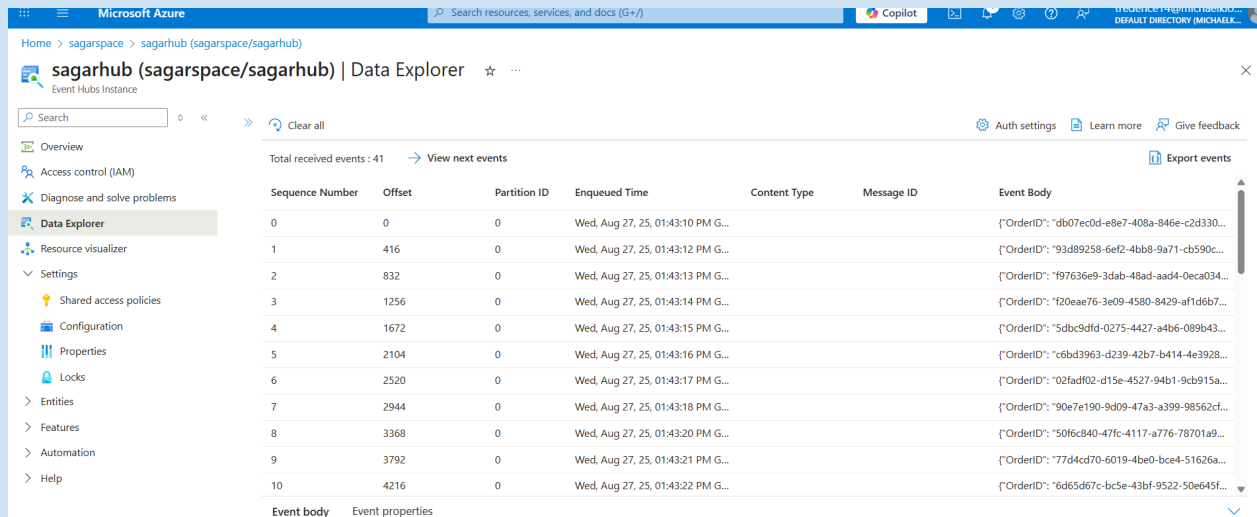
producer = EventHubProducerClient.from_connection_string(conn_str=connection_str, eventhub_name=eventhub_name)

# Stream data row by row
for _, row in df.iterrows():
    data = row.to_dict()
    event_data_batch = producer.create_batch()
    event_data_batch.add(EventData(json.dumps(data)))
    producer.send_batch(event_data_batch)
    print(f'Sent: {data["OrderID"]}')
    time.sleep(1) # simulate 1 event/sec
```

sent file:

```
WARNING: The script faker is installed in '/home/sqladmin/.local/bin' which is not
Consider adding this directory to PATH or, if you prefer to suppress this warning,
Successfully installed faker-37.5.3 numpy-2.3.2 pandas-2.3.2 tzdata-2025.2
sqladmin@sagarvm:~$ python3 send.py
Sent: a1f3d6b7-45e8-4a30-8e9b-1f9d2caa6710
Sent: b7d4f3c2-9c23-4a0a-a4b5-f0317b53f8f5
Sent: c6e8f5a1-58a9-4980-a71c-34266c5db412
Sent: d8f4c9b2-92d2-4a9f-a6b0-15db9d214e62
Sent: e3c9a2f1-73a4-41e8-b4fd-62b0a930ed7b
Sent: f5d6b8c3-84b9-4a77-a9c1-9d1e1cbf6d2e
Sent: g7a2d1e4-91c5-4f21-bf28-7bc0e55d9183
Sent: h9f8e3d5-11a2-48a4-9c2b-64b0b11c6d45
Sent: i2d4b7a6-5c31-4c29-bd38-27f7a04d0c9e
```

## File received on EH



The screenshot shows the Microsoft Azure Data Explorer interface. The top navigation bar includes the Microsoft Azure logo, a search bar, and a Copilot button. The main header displays 'sagarhub (sagarspace/sagarhub) | Data Explorer'. Below the header, there's a search bar and a 'Clear all' button. The left sidebar contains a navigation menu with options like Overview, Access control (IAM), Diagnose and solve problems, Data Explorer (selected), Resource visualizer, Settings, Shared access policies, Configuration, Properties, Locks, Entities, Features, Automation, and Help. The main content area shows a table of events with columns: Sequence Number, Offset, Partition ID, Enqueued Time, Content Type, Message ID, and Event Body. The table lists 11 events, each with a unique OrderID and a timestamp from Wednesday, August 27, 2025, at 01:43:10 PM to 01:43:22 PM. The Event Body column contains JSON-like strings representing IoT telemetry data.

| Sequence Number | Offset | Partition ID | Enqueued Time                     | Content Type | Message ID | Event Body                                        |
|-----------------|--------|--------------|-----------------------------------|--------------|------------|---------------------------------------------------|
| 0               | 0      | 0            | Wed, Aug 27, 25, 01:43:10 PM G... |              |            | {"OrderID": "db07ec0d-e8e7-408a-846e-c2d330..."}  |
| 1               | 416    | 0            | Wed, Aug 27, 25, 01:43:12 PM G... |              |            | {"OrderID": "93d89258-6ef2-4bb8-9a71-cb590c..."}  |
| 2               | 832    | 0            | Wed, Aug 27, 25, 01:43:13 PM G... |              |            | {"OrderID": "f97636e9-3dab-48ad-aad4-0eca034..."} |
| 3               | 1256   | 0            | Wed, Aug 27, 25, 01:43:14 PM G... |              |            | {"OrderID": "f20eae76-3e09-4580-8429-af1d6b7..."} |
| 4               | 1672   | 0            | Wed, Aug 27, 25, 01:43:15 PM G... |              |            | {"OrderID": "5dbc9dfd-0275-4427-a4b6-089b43..."}  |
| 5               | 2104   | 0            | Wed, Aug 27, 25, 01:43:16 PM G... |              |            | {"OrderID": "c6bd3963-d239-42b7-b414-4e3928..."}  |
| 6               | 2520   | 0            | Wed, Aug 27, 25, 01:43:17 PM G... |              |            | {"OrderID": "02fadf02-d15e-4527-94b1-9cb915a..."} |
| 7               | 2944   | 0            | Wed, Aug 27, 25, 01:43:18 PM G... |              |            | {"OrderID": "90e7e190-9d09-47a3-a399-98562cf..."} |
| 8               | 3368   | 0            | Wed, Aug 27, 25, 01:43:20 PM G... |              |            | {"OrderID": "50f6c840-47fc-4117-a776-78701a9..."} |
| 9               | 3792   | 0            | Wed, Aug 27, 25, 01:43:21 PM G... |              |            | {"OrderID": "77d4cd70-6019-4be0-bce4-51626a..."}  |
| 10              | 4216   | 0            | Wed, Aug 27, 25, 01:43:22 PM G... |              |            | {"OrderID": "6d65d67c-bc5e-43bf-9522-50e645f..."} |

Reading data from eventhub and storing into my bronze schema as `iot_telemetry`

#receiving data from Eventhub

```
event_hub_connection_string = (
```

```
)
```

```
eh_conf = {
```

```
    'eventhubs.connectionString': event_hub_connection_string
```

```
}
```

```
from pyspark.sql.types import *
```

```
from pyspark.sql.functions import *
```

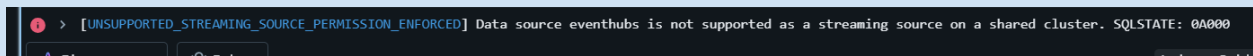
```
schema = StructType([
    StructField("OrderID", StringType()),
    StructField("CustomerID", StringType()),
    StructField("Region", StringType()),
    StructField("StoreID", StringType()),
    StructField("Product", StringType()),
    StructField("Quantity", IntegerType()),
    StructField("UnitPrice", DoubleType()),
    StructField("SalesAmount", DoubleType()),
    StructField("OrderTime", StringType()),
    StructField("DeviceType", StringType()),
    StructField("Temperature", DoubleType()),
    StructField("DeviceStatus", StringType()),
    StructField("AnomalyFlag", IntegerType()),
    StructField("AnomalyType", StringType())
```

))

```
stream_df = (  
    spark.readStream  
        .format("eventhubs")  
        .options(**eh_conf)  
        .load()  
)
```

```
telemetry_df = (  
    stream_df  
        .selectExpr("cast(body as string) as json")  
        .select(from_json("json", schema).alias("data"))  
        .select("data.*")  
)  
telemetry_df.write.mode('append').saveAsTable('iot_telemetry')  
display(telemetry_df)
```

error i got->



i solved by making personal cluster .

Data streamed in from event\_hub->

The screenshot shows a Databricks workspace with a Python notebook on the left and a table view on the right. The notebook contains the following code:

```
telemetry_df = (  
    stream_df  
        .selectExpr("cast(body as string) as json")  
        .select(from_json("json", schema).alias("data"))  
        .select("data.*")  
)  
telemetry_df.write.mode('append').saveAsTable('iot_telemetry')  
display(telemetry_df)
```

The table view displays the following data:

| OrderID                              | CustomerID | Region | StoreID   | Product    | Quantity | UnitPrice |
|--------------------------------------|------------|--------|-----------|------------|----------|-----------|
| 77d4cd70-6019-4be0-bce4-51626ae52101 | CUST-8517  | West   | Store-009 | Mobile     | 2        | 1502.73   |
| 6d65d87c-bc5e-43bf-9522-50e645af58f8 | CUST-7543  | North  | Store-008 | Mobile     | 5        | 1012.36   |
| db3b3f11-076e-4c24-a836-318d6588eedd | CUST-7916  | East   | Store-013 | Tablet     | 5        | 1992.44   |
| 18175bf8-6017-413a-9e20-c876657ee103 | CUST-1188  | East   | Store-018 | Printer    | 3        | 1548.77   |
| 54a45330-e78f-4d3d-9426-8eb38292bdc3 | CUST-1053  | North  | Store-017 | Printer    | 2        | 1039.98   |
| f40b3d98-1113-4b72-9a70-3e1dca06544  | CUST-4258  | West   | Store-012 | Printer    | 2        | 1101.79   |
| f8d9eb0c-6447-4cf0-8e46-cbb6a04d94e4 | CUST-2832  | North  | Store-010 | Mobile     | 1        | 519.7     |
| 1131c959-e455-4c4d-91c2-d5754ab7ad1d | CUST-2133  | West   | Store-005 | Smartwatch | 4        | 1896.35   |
| 30767b0c-128e-46e0-ad00-89fa1ff2ea3  | CUST-4470  | West   | Store-010 | Tablet     | 3        | 904.3     |
| a4efc082-40d2-40cc-8b05-07d3f3259e94 | CUST-6539  | East   | Store-019 | Camera     | 2        | 1197.46   |
| cc0ff1ab-74fe-49f5-ba0d-ect9386d4d40 | CUST-6413  | East   | Store-017 | Mobile     | 3        | 1354.51   |
| a19e0f19-3461-45de-b5e7-f706132d2afc | CUST-8744  | West   | Store-016 | Printer    | 4        | 421.3     |
| 6669123b-763f-420a-90c2-651e8fc3f3f0 | CUST-8651  | East   | Store-004 | Laptop     | 4        | 1470.02   |
| fc349f9b-2650-4e07-b1eb-af4d7d6b1922 | CUST-3296  | South  | Store-006 | Headphones | 4        | 537.12    |

Both table from adls and eventhub stored in my bronze schema of catalog.

iii) Azure synapse ingestion ->

# Synapse connection details

synapse\_workspace =

```

synapse_pool =
synapse_user =
synapse_password =
synapse_jdbc_url =

temp_storage_account =
temp_container =
temp_dir =
f"abfss://{temp_container}@{temp_storage_account}.dfs.core.windows.net/synapse-d
bx-staging"

# Source and Target table names
source_table =
target_catalog =
target_schema =
target_table_name = f"{target_catalog}.{target_schema}.iot_syn"

print(f"Reading data from Synapse table: {source_table}...")
df = (spark.read
    .format("com.microsoft.sqlserver.jdbc.spark") # Use the Synapse connector
    .option("url", synapse_jdbc_url)
    .option("dbtable", source_table)
    .option("user", synapse_user)
    .option("password", synapse_password)
    .option("tempDir", temp_dir) # Crucial for performance
    .option("forwardSparkAzureStorageCredentials", "true")
    .load()
)

# Write the data as a Delta table in Databricks
print(f"Writing data to Delta table: {target_table_name}...")
(df.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable(target_table_name)
)

print("Import complete!")

```

```
display(spark.table('sagar_cat_project1.bronze.iot_syn'))
```

► (4) Spark Jobs

► adls\_ingested: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 12 more fields]

|    | OrderID                               | CustomerID | Region | StoreID   | Product    | Quantity |
|----|---------------------------------------|------------|--------|-----------|------------|----------|
| 57 | 9a72be7d-3ca5-4b64-878c-f443aea6cae5  | CUST-4613  | South  | Store-008 | Headphones | 4        |
| 58 | 9fad8b36-fcd2-4037-a019-728e16ca3686  | CUST-3704  | South  | Store-005 | Camera     | 13       |
| 59 | 199de6e5-8d11-4887-b2b7-c05dab1b01... | CUST-3223  | South  | Store-006 | Laptop     | 3        |
| 60 | 8b67fa52-42bb-486a-866c-aa1a893ee151  | CUST-7211  | North  | Store-014 | Headphones | 1        |
| 61 | 3afae91f-cc69-46ec-941b-58cefc85d7e   | CUST-2124  | East   | Store-001 | Mobile     | 2        |
| 62 | fae848cf-90c7-46fe-af09-87e58efbf673  | CUST-4571  | South  | Store-009 | Printer    | 3        |
| 63 | 14dc706f-5b8c-45ff-98a3-5246959b1b69  | CUST-3683  | North  | Store-004 | Camera     | 1        |
| 64 | 2b4da48a-9939-4bb3-8203-0f35369dff71  | CUST-4249  | East   | Store-019 | Smartwatch | 2        |
| 65 | 2bb8c321-2f07-408d-895a-217cda90ee50  | CUST-1672  | North  | Store-018 | Tablet     | 3        |
| 66 | 82961f57-063d-452e-abe1-c8c0ce076f1d  | CUST-2729  | South  | Store-012 | Smartwatch | 4        |
| 67 | cd669277-e660-44cd-82ad-a365c6d375cd  | CUST-5425  | South  | Store-015 | Tablet     | 5        |
| 68 | 22c7aca4-9dda-4759-b9b9-2c585ae3cc71  | CUST-8613  | South  | Store-011 | Laptop     | 4        |
| 69 | 37ce4b04-72d4-4b8e-ad0f-0d98e56445a0  | CUST-6576  | North  | Store-020 | Smartwatch | 3        |
| 70 | 5e8bc1da-b338-4da5-a533-e79d55161bc9  | CUST-9004  | West   | Store-016 | Smartwatch | 4        |
| 71 |                                       |            |        |           |            |          |

## Transformation:

### # Removal of Data Quality issues :

#data cleaning and deduplication  
from pyspark.sql import functions as F

# Load bronze tables

```
df_iot = spark.table("sagar_cat_project1.bronze.adls_ingested_iot")
```

```
df_sales = spark.table("sagar_cat_project1.bronze.iot_telemetry")
```

# Handle nulls (example: replace missing Product/DeviceType with "Unknown")

```
df_sales = df_sales.fillna({"Product": "Unknown"})
```

```
df_iot = df_iot.fillna({"DeviceStatus": "UNKNOWN"})
```

# Deduplicate



```
df_sales = df_sales.dropDuplicates(["OrderID"])
df_iot = df_iot.dropDuplicates(["OrderID", "DeviceType", "OrderTime"])
```

## # Join IoT telemetry with Sales Transactions

# Convert OrderTime to timestamp

```
df_sales = df_sales.withColumn("OrderTime", F.to_timestamp("OrderTime"))
df_iot = df_iot.withColumn("OrderTime", F.to_timestamp("OrderTime"))
```

# Join: same store, same order timestamp

```
df_joined = df_sales.alias("s") \
    .join(df_iot.alias("i"),
          (F.col("s.StoreID") == F.col("i.StoreID")) &
          (F.col("s.OrderTime") == F.col("i.OrderTime")),
          "inner")
```

```
import pyspark.sql.functions as F
from itertools import chain
```

```
df_final = df_joined.withColumn(
    "AnomalyFlag_Sales",
    F.when(F.col("i.AnomalyFlag") == 1, 1).otherwise(0)
)
```

# Normalize store IDs (e.g., "Store-001" → 1)

```
df_final = df_final.withColumn(
    "StoreID_norm",
    F.regexp_replace(F.col("s.StoreID"), "Store-", "").cast("int")
)
```

# Normalize product categories

# The map should be between products and mentioned product categories .Any product not fitting these categories will result in `null`, avoiding mix-ups or wrong classifications.

```
product_map = {
    "Mobile": "Electronics",
    "Laptop": "Electronics",
    "Tablet": "Electronics",
    "Printer": "Peripherals",
    "Smartwatch": "Wearables"
}
```

```
mapping_expr = F.create_map([F.lit(x) for x in chain(*product_map.items())])
```

```
df_final = df_final.withColumn(
    "ProductCategory",
    mapping_expr[F.col("s.Product")]
)
```

```
df_final.select("s.StoreID", "StoreID_norm", "s.Product", "ProductCategory",
    "i.AnomalyFlag", "AnomalyFlag_Sales").show()
```

The screenshot shows the Databricks interface with a Python notebook. The code in the notebook is as follows:

```
# A slightly cleaner way to create the mapping expression
mapping_expr = F.create_map([F.lit(x) for x in chain(*product_map.items())])

df_final = df_final.withColumn(
    "ProductCategory",
    mapping_expr[F.col("s.Product")]
)

df_final.select("s.StoreID", "StoreID_norm", "s.Product", "ProductCategory", "i.AnomalyFlag", "AnomalyFlag_Sales").show()
```

The output of the code is a table with the following columns: StoreID, StoreID\_norm, Product, ProductCategory, i.AnomalyFlag, and AnomalyFlag\_Sales. The table contains 20 rows of data, including products like Headphones, Camera, Laptop, Tablet, Smartwatch, Printer, and Wearables.

| StoreID     | StoreID_norm | Product    | ProductCategory | i.AnomalyFlag | AnomalyFlag_Sales |
|-------------|--------------|------------|-----------------|---------------|-------------------|
| [Store-003] | 3            | Headphones | NULL            | 0             | 0                 |
| [Store-020] | 20           | Camera     | NULL            | 1             | 1                 |
| [Store-007] | 7            | Laptop     | Electronics     | 0             | 0                 |
| [Store-019] | 19           | Tablet     | Electronics     | 0             | 0                 |
| [Store-001] | 1            | Smartwatch | Wearables       | 0             | 0                 |
| [Store-004] | 4            | Headphones | NULL            | 0             | 0                 |
| [Store-006] | 6            | Headphones | NULL            | 1             | 1                 |
| [Store-007] | 7            | Smartwatch | Wearables       | 0             | 0                 |
| [Store-019] | 19           | Printer    | Peripherals     | 0             | 0                 |
| [Store-006] | 6            | Headphones | NULL            | 0             | 0                 |
| [Store-001] | 1            | Laptop     | Electronics     | 0             | 0                 |
| [Store-020] | 20           | Printer    | Peripherals     | 0             | 0                 |
| [Store-015] | 15           | Smartwatch | Wearables       | 0             | 0                 |
| [Store-018] | 18           | Tablet     | Electronics     | 0             | 0                 |
| [Store-009] | 9            | Smartwatch | Wearables       | 0             | 0                 |
| [Store-015] | 15           | Tablet     | Electronics     | 1             | 1                 |
| [Store-010] | 10           | Headphones | NULL            | 1             | 1                 |
| [Store-004] | 4            | Headphones | NULL            | 0             | 0                 |

# Saving my Silver Table:

```
# MAGIC %sql
# MAGIC use catalog sagar_cat_project1;
# MAGIC use schema silver;
# MAGIC drop table if exists iot_cleaned;
```

```
import pyspark.sql.functions as F
```

```
df_cleaned = df_final.select(
    F.col("s.OrderID").alias("OrderID"),
    F.col("s.CustomerID").alias("CustomerID"),
    "StoreID_norm",
    "ProductCategory",
    F.col("s.Region").alias("Region"),
    F.col("s.Quantity").alias("Quantity"),
    F.col("s.UnitPrice").alias("UnitPrice"),
    F.col("s.SalesAmount").alias("SalesAmount"),
    F.col("s.OrderTime").alias("OrderTime"),
    F.col("i.DeviceType").alias("DeviceType"),
    F.col("i.Temperature").alias("Temperature"),
    F.col("i.DeviceStatus").alias("DeviceStatus"),
    "AnomalyFlag_Sales",
    F.col("i.AnomalyType").alias("AnomalyType")
)
```

```
df_cleaned.write.format("delta").mode("overwrite").option("mergeSchema",
"true").saveAsTable(
    "sagar_cat_project1.silver.iot_cleaned"
)
```

## # Partitioning, Z-order and Optimize

```
from pyspark.sql import functions as F
CATALOG = "sagar_cat_project1"
SCHEMA = "silver"
TABLE_NAME = "iot_cleaned"
FULL_TABLE_NAME = f'{CATALOG}.{SCHEMA}.{TABLE_NAME}'
```

```
# Load the existing Silver table
df = spark.table(FULL_TABLE_NAME)
```

```
# Create a date column from the OrderTime timestamp to partition by date
if "order_date" not in df.columns:
```

```

df = df.withColumn("order_date", F.to_date("OrderTime"))

# Standardize StoreID column if necessary
if "StoreID_norm" in df.columns and "StoreID" not in df.columns:
    df = df.withColumnRenamed("StoreID_norm", "StoreID")

(df.write
 .format("delta")
 .mode("overwrite")
 .option("overwriteSchema", "true")
 .partitionBy("Region", "order_date") # Apply partitioning
 .saveAsTable('sagar_cat_project1.silver.iot_cleaned_zorder')
)

print("Table successfully rewritten with new partitions.")

# Apply Z-Ordering via the OPTIMIZE command
# This step physically reorganizes the data within the new partitions
print("Applying Z-Ordering on StoreID and ProductCategory...")

# The OPTIMIZE command now runs on the newly partitioned table
spark.sql(f"""
    OPTIMIZE {'sagar_cat_project1.silver.iot_cleaned_zorder'}
    ZORDER BY (StoreID, ProductCategory)
    """)

print("Z-Ordering complete.")

# (Optional but Recommended) Verify the changes
print("\nVerifying table properties...")
display(spark.sql(f"DESCRIBE DETAIL {FULL_TABLE_NAME}"))

```

```
from pyspark.sql import functions as F

# 1. Configuration
CATALOG = "sagar_cat_project1"
SCHEMA = "silver"
TABLE_NAME = "iot_cleaned"
FULL_TABLE_NAME = f"{CATALOG}.{SCHEMA}.{TABLE_NAME}"

# Load the existing Silver table
df = spark.table(FULL_TABLE_NAME)

(df.write
 .format("delta")
 .mode("overwrite")
 .option("overwriteSchema", "true")
 .partitionBy("Region") # Apply partitioning by Region
 .saveAsTable(f"sagar_cat_project1.silver.iot_cleaned_partitionbyregion"))

print("Table successfully rewritten and partitioned by Region.")

# 3. (Recommended) Verify the partitioning
print("\nVerifying table properties...")
display(spark.sql(f"DESCRIBE DETAIL {FULL_TABLE_NAME}"))
> See performance \(3\)
```

df: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 12 more fields]

Table successfully rewritten and partitioned by Region.

Verifying table properties...

| Table | format | id                                    | name                                   | description | location                                        |
|-------|--------|---------------------------------------|----------------------------------------|-------------|-------------------------------------------------|
| 1     | delta  | 29904b60-4ade-465d-a47e-e03496ec33... | sagar_cat_project1.silver.iot_clean... | null        | > abfss://raw8@sagarcatalog.dfs.core.windows... |

Have a look on all the tables that is created in silver region

iot\_cleaned->

display(spark.table(full\_table\_name).limit(10))

> [See performance \(3\)](#)

--- Displaying top 5 rows of: sagar\_cat\_project1.silver.iot\_cleaned ---

|    | OrderID                               | CustomerID | StoreID | ProductCategory | Region | Quantity | UnitPrice | SalesAmount | OrderTime                     | DeviceType   | Temperature | DeviceStatus | AnomalyFlag | Sales | AnomalyType  |
|----|---------------------------------------|------------|---------|-----------------|--------|----------|-----------|-------------|-------------------------------|--------------|-------------|--------------|-------------|-------|--------------|
| 1  | 655827a-9eaf-4261-948f-3c5f02adcd4    | CUST-9476  | 20      | Wearables       | East   | 3        | 633.22    | 2499.66     | 2023-01-08T23:27:07.000+00... | POS_Terminal | 16.43       | OK           | 0           |       |              |
| 2  | 9142069-91ca-4699-8016-cd3805daab     | CUST-2373  | 20      | Wearables       | North  | 9        | 592.72    | 4974.48     | 2023-01-01T06:22:20.000+00... | Temp_Sensor  | 23.21       | MISMATCH     | 1           |       | RFD_Mismatch |
| 3  | 010e0b0-61aa-4969-aee5-1317b31727...  | CUST-2860  | 7       | Electronics     | West   | 2        | 1604.24   | 3208.48     | 2023-01-16T01:46:21.000+00... | POS_Terminal | 21.4        | OK           | 0           |       |              |
| 4  | 7623386-58ac-433d-8993-2071c10e9...   | CUST-5044  | 19      | Electronics     | North  | 4        | 952.26    | 3809.04     | 2023-01-02T12:15:13.000+00... | Temp_Sensor  | 18.64       | OK           | 0           |       |              |
| 5  | 28b37595-205a-422a-8953-cd71a9f80a    | CUST-4842  | 1       | Wearables       | East   | 1        | 1818.42   | 1818.42     | 2023-01-28T20:56:11.000+00... | Temp_Sensor  | 21.49       | OK           | 0           |       |              |
| 6  | 18c3a44-10b0-43af-8a65-5d219a0e194    | CUST-2843  | 4       | Wearables       | West   | 1        | 447.28    | 447.28      | 2023-01-14T19:01:10.000+00... | RFD_Scanner  | 23.84       | OK           | 0           |       |              |
| 7  | 046e234b-5477-4dfe-8b47-5f83a3d9e92   | CUST-2450  | 6       | Wearables       | South  | 10       | 716.98    | 7169.8      | 2023-01-15T10:36:04.000+00... | RFD_Scanner  | 18.68       | MISMATCH     | 1           |       | RFD_Mismatch |
| 8  | 7794056-b6a3-409d-804a-d9d679a6d84    | CUST-2609  | 7       | Wearables       | West   | 1        | 945.77    | 945.77      | 2023-01-26T09:34:33.000+00... | RFD_Scanner  | 18.03       | OK           | 0           |       |              |
| 9  | b272890-3a6a-46a3-4795-476864101b     | CUST-9666  | 19      | Peripherals     | North  | 3        | 1125.51   | 3376.53     | 2023-01-30T11:45:01.000+00... | RFD_Scanner  | 24.28       | OK           | 0           |       |              |
| 10 | bc9e9732-838f-4153-6894-768557a33c... | CUST-1472  | 6       | Wearables       | West   | 5        | 76.67     | 383.35      | 2023-01-20T19:20:36.000+00... | RFD_Scanner  | 19.57       | OK           | 0           |       |              |

10 rows | 2.25s runtime

--- Displaying top 5 rows of: sagar\_cat\_project1.silver.iot\_cleaned\_partitionbyregion ---

after partitioning->

--- Displaying top 5 rows of: `sagar_cat_project1.silver.iot_cleaned_partitionbyregion` ---

|    | $\Delta_1$ OrderID                      | $\Delta_1$ CustomerID | $\Delta_1$ StoreID_norm | $\Delta_1$ ProductCategory | $\Delta_1$ Region | $\Delta_1$ Quantity | 1.2 UnitPrice | 1.2 SalesAmount | $\Delta_1$ OrderTime          | $\Delta_1$ DeviceType | 1.2 Temperature | $\Delta_1$ DeviceStatus | $\Delta_1$ AnomalyFlag_Sales | $\Delta_1$ AnomalyType |
|----|-----------------------------------------|-----------------------|-------------------------|----------------------------|-------------------|---------------------|---------------|-----------------|-------------------------------|-----------------------|-----------------|-------------------------|------------------------------|------------------------|
| 1  | 0104080-819a-4993-ae5c-1917b3717127...  | CUST-2660             |                         | 7 Electronics              | West              | 2                   | 1024.24       | 2058.48         | 2023-01-16T01:45:21.000+00... | POS_Terminal          | 21.4            | OK                      | 0                            | None                   |
| 2  | 1b3d454-15b-4aaf-a6d5-5210a3c9c374...   | CUST-2663             |                         | 4 [REDACTED]               | West              | 1                   | 447.28        | 447.28          | 2023-01-14T09:10:00.000+00... | RFD_Scanner           | 23.84           | OK                      | 0                            | None                   |
| 3  | 7704956-9e0b-4c98-809a-d9d76949d8f4...  | CUST-2609             |                         | 7 Wearables                | West              | 1                   | 945.77        | 945.77          | 2023-01-26T08:43:33.000+00... | RFD_Scanner           | 18.02           | OK                      | 0                            | None                   |
| 4  | bc9a732-838f-4153-6896-78357f313c...    | CUST-1472             |                         | 6 [REDACTED]               | West              | 5                   | 76.67         | 383.35          | 2023-01-20T19:03:36.000+00... | RFD_Scanner           | 19.57           | OK                      | 0                            | None                   |
| 5  | a19d46b-14c9-43ce-a809-ba6900c93140...  | CUST-5636             |                         | 18 Electronics             | West              | 1                   | 305.26        | 305.26          | 2023-01-29T04:11:45.000+00... | RFD_Scanner           | 23.95           | OK                      | 0                            | None                   |
| 6  | e6b59589-8152-418f-9666-ae51b777e611... | CUST-4259             |                         | 10 [REDACTED]              | West              | 10                  | 1362.73       | 1362.73         | 2023-01-24T15:58:16.000+00... | Temp_Sensor           | 21.13           | MISMATCH                | 1                            | RFD_Mismatch           |
| 7  | 2be1ee8-d822-4c83-8f93-dbe119400020...  | CUST-4610             |                         | 11 Electronics             | West              | 3                   | 1057.18       | 3171.54         | 2023-01-01T11:19:42.000+00... | Temp_Sensor           | 23.83           | OK                      | 0                            | None                   |
| 8  | 3076780c-128e-46e6-a8db-899a1f5eeea3... | CUST-4470             |                         | 10 Electronics             | West              | 3                   | 904.3         | 2712.9          | 2023-01-26T03:01:10.000+00... | POS_Terminal          | 18.85           | OK                      | 0                            | None                   |
| 9  | 451212d0-6a42-49c6-a854-5766b76275...   | CUST-5056             |                         | 10 [REDACTED]              | West              | 1                   | 1625.26       | 1625.26         | 2023-01-04T04:18:47.000+00... | POS_Terminal          | 18.8            | OK                      | 0                            | None                   |
| 10 | ad7819a2-179e-4845-9648-933a72a6f532... | CUST-2946             |                         | 4 Electronics              | West              | 3                   | 1600.05       | 4800.15         | 2023-01-17T11:03:46.000+00... | Temp_Sensor           | 24.57           | OK                      | 0                            | None                   |

10 rows | 2.25s runtime

partitioning zorder->

--- Displaying top 5 rows of: `sagar_cat_project1.silver.iot_cleaned_zorder` ---

|    | $\Delta_1$ OrderID                      | $\Delta_1$ CustomerID | $\Delta_1$ StoreID_norm | $\Delta_1$ ProductCategory | $\Delta_1$ Region | $\Delta_1$ Quantity | 1.2 UnitPrice | 1.2 SalesAmount | $\Delta_1$ OrderTime          | $\Delta_1$ DeviceType | 1.2 Temperature | $\Delta_1$ DeviceStatus | $\Delta_1$ AnomalyFlag_Sales | $\Delta_1$ AnomalyType |
|----|-----------------------------------------|-----------------------|-------------------------|----------------------------|-------------------|---------------------|---------------|-----------------|-------------------------------|-----------------------|-----------------|-------------------------|------------------------------|------------------------|
| 1  | 323d3d1e-15ec-4ff5-bc94-18f8d7f068d...  | CUST-5643             |                         | 2 [REDACTED]               | West              | 3                   | 652.34        | 1957.02         | 2023-01-06T23:50:45.000+00... | POS_Terminal          | 24.98           | OK                      | 0                            | None                   |
| 2  | ac3193e5-887e-48a3-b8ad-d97ab3e3d8...   | CUST-4066             |                         | 6 Electronics              | West              | 5                   | 555.67        | 2778.35         | 2023-01-06T03:53:03.000+00... | POS_Terminal          | 35.33           | OVERHEAT                | 1                            | Overheat               |
| 3  | 93205e99-7804-415c-be50-d74d78e70a3...  | CUST-3960             |                         | 1 [REDACTED]               | West              | 4                   | 1611          | 6444            | 2023-01-06T23:21:28.000+00... | RFD_Scanner           | 18.9            | OK                      | 0                            | None                   |
| 4  | 4210a853-d866-4a09-915c-4560837981...   | CUST-3598             |                         | 17 Electronics             | West              | 1                   | 708.58        | 708.58          | 2023-01-06T12:47:01.000+00... | Temp_Sensor           | 19.59           | OK                      | 0                            | None                   |
| 5  | b7ca0e0-a5b9-4a05-a65a-9588734a20...    | CUST-5748             |                         | 19 Wearables               | West              | 13                  | 710.43        | 9235.59         | 2023-01-06T10:51:25.000+00... | Temp_Sensor           | 21.35           | MISMATCH                | 1                            | RFD_Mismatch           |
| 6  | e761c1c6-5ac1-484f-9707-954472d2c64...  | CUST-2776             |                         | 18 Peripherals             | West              | 4                   | 475.85        | 1903.4          | 2023-01-06T03:47:23.000+00... | POS_Terminal          | 23.01           | OK                      | 0                            | None                   |
| 7  | e139a9e6-a169-4779-82b1-c687f6ee83...   | CUST-5291             |                         | 15 [REDACTED]              | West              | 4                   | 1143.77       | 4570.08         | 2023-01-06T19:20:33.000+00... | RFD_Scanner           | 24.82           | OK                      | 0                            | None                   |
| 8  | 718a4d55-78e9-4a4b-819a-ba3597b3e4f...  | CUST-5493             |                         | 6 Peripherals              | West              | 2                   | 394.85        | 789.9           | 2023-01-06T16:13:06.000+00... | RFD_Scanner           | 24.96           | OK                      | 0                            | None                   |
| 9  | 2a32a63-3188-49a2-8963-38182b4524b6...  | CUST-1811             |                         | 20 Wearables               | West              | 1                   | 1430.75       | 1430.75         | 2023-01-06T19:47:06.000+00... | RFD_Scanner           | 24              | OK                      | 0                            | None                   |
| 10 | e8569a07-c399-4876-836a-c1d5d5c3d09e... | CUST-6405             |                         | 3 Electronics              | North             | 4                   | 220.25        | 881             | 2023-01-19T23:10:41.000+00... | RFD_Scanner           | 20.82           | OK                      | 0                            | None                   |

10 rows | 2.25s runtime

## Aggregation & Enrichment Layer (Gold)->

### Gold Layer (Aggregation & Enrichment) – Summary

- **Source:** Cleaned Silver table (`iot_cleaned`)
- **Schema Prep:** Renamed columns for consistency (`sales_amount` → `SalesAmount`, `StoreID_norm` → `StoreID`), casted data types, derived `order_date` & `order_month`.

### KPIs & Aggregations:

1. **Sales per Region:** Daily & monthly totals (`total_sales`, `total_units_sold`, `distinct_orders`)
2. **Device Anomaly Trend:** Daily anomaly count & rate per store
3. **Conversion Impact:** Success rate of orders vs. anomalies per store/day

### Enrichments:

4. **Performance Tiers:** Stores ranked monthly by sales, tagged `High/Medium/Low`
5. **Weighted Performance Score:** Min-max normalized sales vs anomalies → final score =  $0.8 \times \text{sales\_norm} - 0.2 \times \text{anomalies\_norm}$

- **Output:** Delta tables in **gold** schema:
  - **daily\_sales\_by\_region**
  - **monthly\_sales\_by\_region**
  - **daily\_anomaly\_trend\_by\_store**
  - **daily\_conversion\_impact\_by\_store**
  - **monthly\_store\_performance\_tiers**
  - **monthly\_store\_weighted\_score**

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

CATALOG = "sagar_cat_project1"
SILVER_SCHEMA = f"{CATALOG}.silver"
GOLD_SCHEMA = f"{CATALOG}.gold"
BASE_TABLE = f"{SILVER_SCHEMA}.iot_cleaned"

# Create the Gold schema if it doesn't exist
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {GOLD_SCHEMA}")

# Load the base Silver table
base_df = spark.table(BASE_TABLE)

# Standardize column names to ensure consistency
# (Handles cases like 'sales_amount' vs 'SalesAmount' and 'StoreID_norm' vs 'StoreID')
if "sales_amount" in base_df.columns and "SalesAmount" not in base_df.columns:
    base_df = base_df.withColumnRenamed("sales_amount", "SalesAmount")
if "StoreID_norm" in base_df.columns and "StoreID" not in base_df.columns:
    base_df = base_df.withColumnRenamed("StoreID_norm", "StoreID")
```

```

# Ensure correct data types for calculations
base_df = (base_df
    .withColumn("OrderTime", F.to_timestamp("OrderTime"))
    .withColumn("SalesAmount", F.col("SalesAmount").cast("double"))
    .withColumn("Quantity", F.col("Quantity").cast("int"))
)

# Derive common date and month grains for aggregation
base_df = (base_df
    .withColumn("order_date", F.to_date("OrderTime"))
    .withColumn("order_month", F.date_trunc("month", "OrderTime"))
)

# -----
# 1) KPI: Daily and Monthly Sales per Region
# -----

print("Creating sales KPIs per region...")

# Daily aggregation
daily_sales = (base_df
    .groupBy("order_date", "Region")
    .agg(
        F.sum("SalesAmount").alias("total_sales"),
        F.sum("Quantity").alias("total_units_sold"),
        F.countDistinct("OrderID").alias("distinct_orders")
    )
)

(daily_sales.write.format("delta").mode("overwrite")
    .saveAsTable(f"{GOLD_SCHEMA}.daily_sales_by_region"))

# Monthly aggregation
monthly_sales = (base_df
    .groupBy("order_month", "Region")
    .agg(
        F.sum("SalesAmount").alias("total_sales"),
        F.sum("Quantity").alias("total_units_sold"),
        F.countDistinct("OrderID").alias("distinct_orders")
    )
)

```



```

)
(monthly_sales.write.format("delta").mode("overwrite")
  .saveAsTable(f"{GOLD_SCHEMA}.monthly_sales_by_region"))

# -----
# 2) KPI: Device Anomaly Trend per Store
# -----
print("Creating device anomaly trend KPI...")

anomaly_trend = (base_df
  .groupBy("order_date", "StoreID")
  .agg(
    F.sum("AnomalyFlag_Sales").alias("anomaly_count"),
    F.count("OrderID").alias("total_transactions")
  )
  .withColumn("anomaly_rate", F.col("anomaly_count") / F.col("total_transactions")))
)
(anomaly_trend.write.format("delta").mode("overwrite")
  .saveAsTable(f"{GOLD_SCHEMA}.daily_anomaly_trend_by_store"))

# -----
# 3) KPI: Conversion Rate Impact from Device Failures
# -----
print("Creating conversion rate impact KPI...")

# We define "conversion" as the ratio of successful orders to total orders at a store-day level
daily_impact = (base_df
  .groupBy("order_date", "StoreID")
  .agg(
    F.sum(F.when(F.col("AnomalyFlag_Sales") == 1, 1).otherwise(0)).alias("anomaly_orders"),
    F.count("OrderID").alias("total_orders")
  )
  .withColumn("successful_orders", F.col("total_orders") - F.col("anomaly_orders"))
  .withColumn("success_rate", F.col("successful_orders") / F.col("total_orders"))
  .withColumn("had_anomaly_day", F.when(F.col("anomaly_orders") > 0, 1).otherwise(0))
)
(daily_impact.write.format("delta").mode("overwrite")
  .saveAsTable(f"{GOLD_SCHEMA}.daily_conversion_impact_by_store"))

```

```

# -----
# 4) Enrichment: Tag Stores into Performance Tiers
# -----

print("Enriching stores with performance tiers...")

# Calculate total sales per store for each month
store_monthly_sales = base_df.groupBy("order_month",
"StoreID").agg(F.sum("SalesAmount").alias("total_sales"))

# Use a window function to rank stores within each month
w_spec = Window.partitionBy("order_month").orderBy(F.col("total_sales").desc())
store_ranks = store_monthly_sales.withColumn("rank", F.percent_rank().over(w_spec))

# Assign tiers based on percentile rank
store_tiers = store_ranks.withColumn("performance_tier",
    F.when(F.col("rank") <= 0.2, "High")
    .when((F.col("rank") > 0.2) & (F.col("rank") <= 0.8), "Medium")
    .otherwise("Low")
)
(store_tiers.select("order_month", "StoreID", "total_sales", "performance_tier")
.write.format("delta").mode("overwrite")
.saveAsTable(f"{GOLD_SCHEMA}.monthly_store_performance_tiers"))

# -----
# 5) Enrichment: Compute Weighted Sales vs. Anomalies Score
# -----

print("Enriching stores with a weighted performance score...")

# Get monthly sales and anomaly counts per store
monthly_kpis = base_df.groupBy("order_month", "StoreID").agg(
    F.sum("SalesAmount").alias("sales"),
    F.sum("AnomalyFlag_Sales").alias("anomalies")
)

# Define a window to normalize scores across all stores within a month
w_monthly = Window.partitionBy("order_month")

```

```

# Calculate min-max normalized scores for sales and anomalies
normalized_kpis = (monthly_kpis
    .withColumn("max_sales", F.max("sales").over(w_monthly))
    .withColumn("min_sales", F.min("sales").over(w_monthly))
    .withColumn("max_anomalies", F.max("anomalies").over(w_monthly))
    .withColumn("min_anomalies", F.min("anomalies").over(w_monthly))
    .withColumn("sales_norm",
        F.when(F.col("max_sales") == F.col("min_sales"), 1.0)
        .otherwise((F.col("sales") - F.col("min_sales")) / (F.col("max_sales") - F.col("min_sales")))
    )
    .withColumn("anomalies_norm",
        F.when(F.col("max_anomalies") == F.col("min_anomalies"), 0.0)
        .otherwise((F.col("anomalies") - F.col("min_anomalies")) / (F.col("max_anomalies") -
F.col("min_anomalies")))
    )
)

# Compute the final weighted score (e.g., 80% weight for sales, 20% for anomalies)
weighted_score = normalized_kpis.withColumn(
    "performance_score",
    (0.8 * F.col("sales_norm")) - (0.2 * F.col("anomalies_norm"))
)

(weighted_score.select("order_month", "StoreID", "sales", "anomalies", "performance_score")
    .write.format("delta").mode("overwrite")
    .saveAsTable(f"{GOLD_SCHEMA}.monthly_store_weighted_score"))

print("\n--- Gold Layer Generation Complete ---")

```

Finally have a look of all the tables that is created.

The screenshot shows the Databricks workspace interface. On the left is a sidebar with navigation options like Workspace, Catalog, and Recent. The main area displays a Python notebook with a table view of data. The table is titled "Displaying contents of: sagar\_cat\_project1.gold.daily\_sales\_by\_region ---". It has 7 columns: order\_date, Region, 1.2 total\_sales, total\_units\_sold, distinct\_orders, and two unnamed columns. The data is as follows:

|   | order_date | Region | 1.2 total_sales   | total_units_sold | distinct_orders |  |
|---|------------|--------|-------------------|------------------|-----------------|--|
| 1 | 2023-01-20 | East   | 30315.19999999999 | 20               | 7               |  |
| 2 | 2023-01-29 | South  | 32773.42999999999 | 28               | 11              |  |
| 3 | 2023-01-02 | East   | 2706.73           | 19               | 7               |  |
| 4 | 2023-01-11 | North  | 29061.52000000000 | 36               | 11              |  |
| 5 | 2023-01-11 | East   | 24138.62999999999 | 25               | 9               |  |

Below the first table is another table view titled "Displaying contents of: sagar\_cat\_project1.gold.monthly\_sales\_by\_region ---". It has 6 columns: order\_month, Region, 1.2 total\_sales, total\_units\_sold, distinct\_orders, and an unnamed column. The data is as follows:

|   | order_month                   | Region | 1.2 total_sales   | total_units_sold | distinct_orders |  |
|---|-------------------------------|--------|-------------------|------------------|-----------------|--|
| 1 | 2023-01-01T00:00:00.000-00... | East   | 776331.6000000001 | 811              | 252             |  |
| 2 | 2023-01-01T00:00:00.000-00... | South  | 704848.18         | 738              | 225             |  |
| 3 | 2023-01-01T00:00:00.000-00... | North  | 955183.64         | 960              | 289             |  |
| 4 | 2023-01-01T00:00:00.000-00... | West   | 769714.7799999996 | 787              | 234             |  |



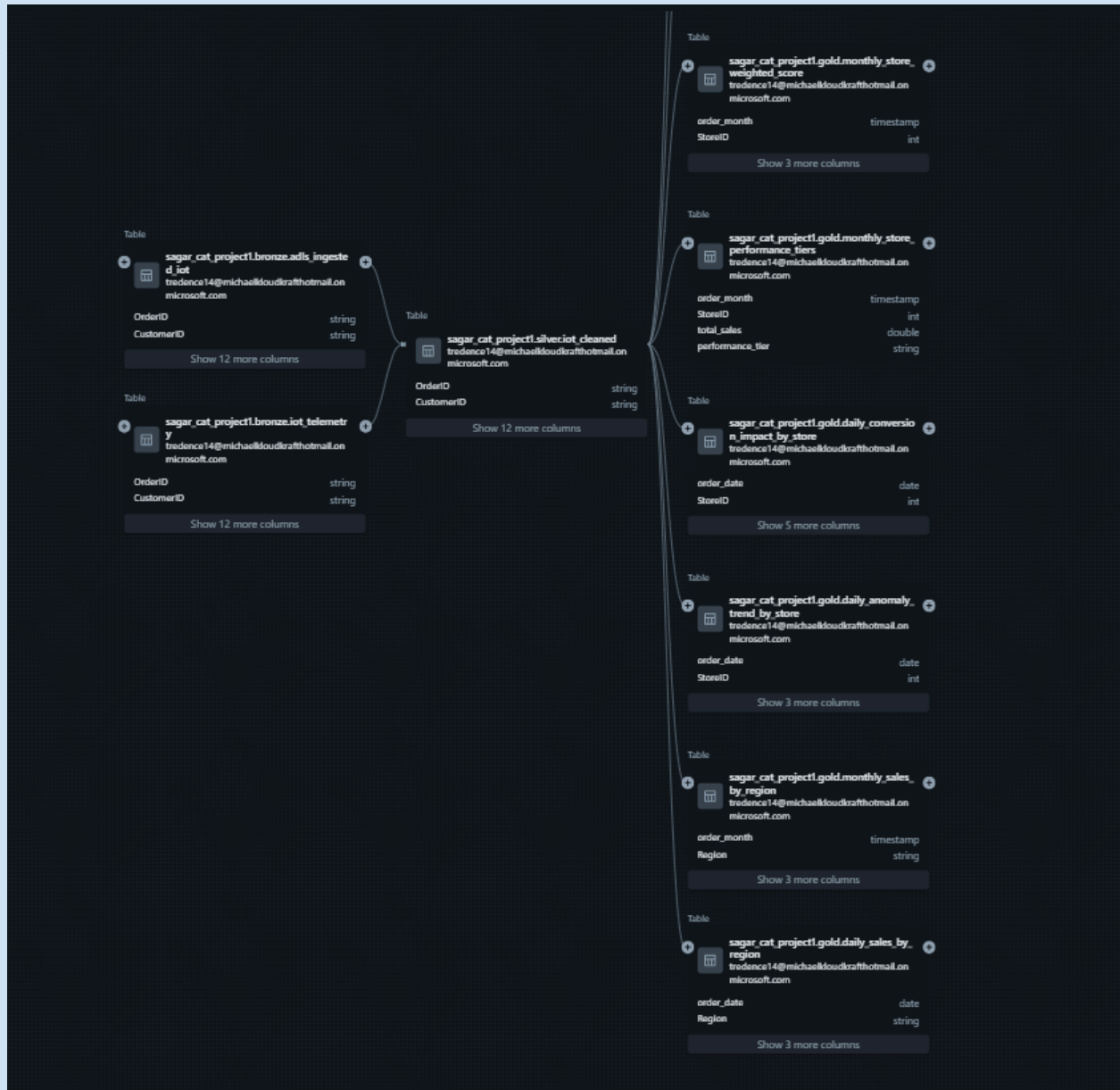
## JOBS creation->

The screenshot displays the Databricks Jobs & Pipelines interface. The main view shows a job named 'sagar\_cap1 run' in the 'List' view. The job consists of three stages: 'Ingest', 'transform', and 'Gold', all of which have succeeded. The 'Job run details' panel on the right shows the job ID, run ID, launch time, and status (Succeeded). The 'Compute' panel shows the job is running on a 'kraft200' node.

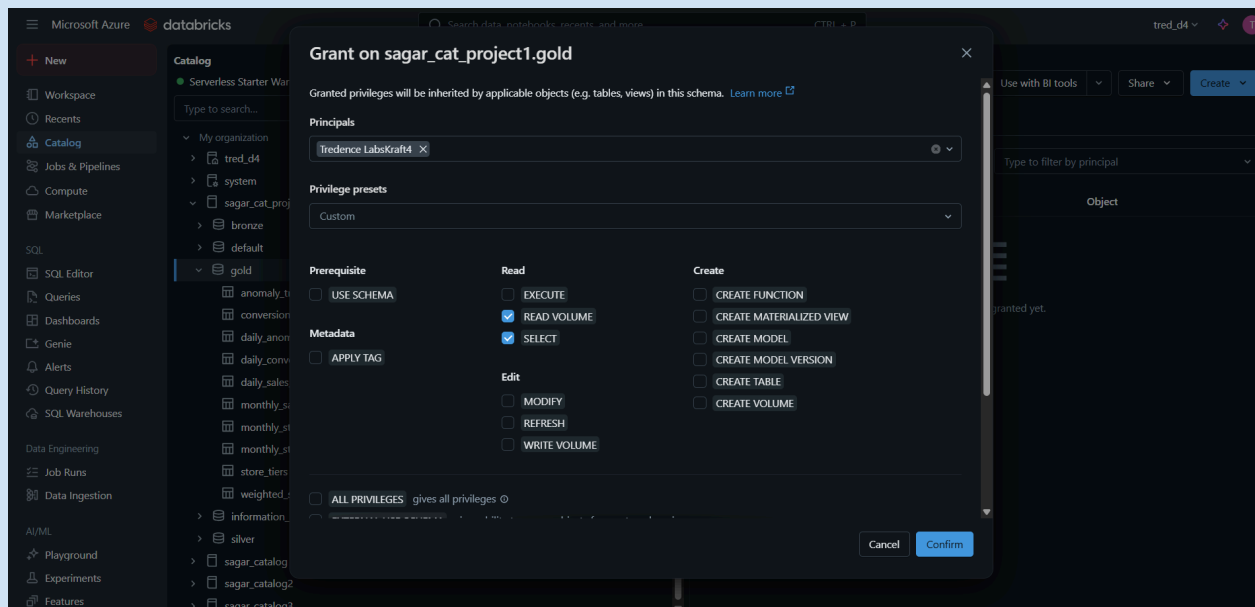
## scheduling for regular ingestion and updation of tables->

The screenshot displays the Databricks Jobs & Pipelines interface with the 'Schedules & Triggers' dialog open for a job named 'sagar\_cap1'. The dialog is set to 'Active' with a 'Scheduled' trigger type. The 'Schedule type' is set to 'Simple' with a 'Periodic' schedule of 'Every 1 Day'. The 'Run now' button is visible in the top right corner.

## Lineage Graph->



Giving bi user access to my gold tables->



Connecting through PowerBi->

